

LOCAL AI CO-BUILDING

Small models. Useful apps.
Built together.

40-minute kickoff

Model and tooling snapshot: 23 September 2026



Today's route

Topic	Clock
1 Why local / edge models	01:30–07:00
2 Finding the right model	07:00–14:30
3 Model type and app architecture	14:30–20:00
4 Three builds, end to end	20:00–35:00
Team brief and build handoff	35:00–40:00

By minute 40: one app idea, a model choice, and a test that defines success.

Why put the model on the device?

Private and Secure: Sensitive inputs

Keep documents, recordings and images on the device.

Work without a connection

Make the core workflow usable after setup and downloads.

Responsive, repeated tasks

Avoid network round trips and reuse the same local model (no unexpected model change.)

Control over the stack

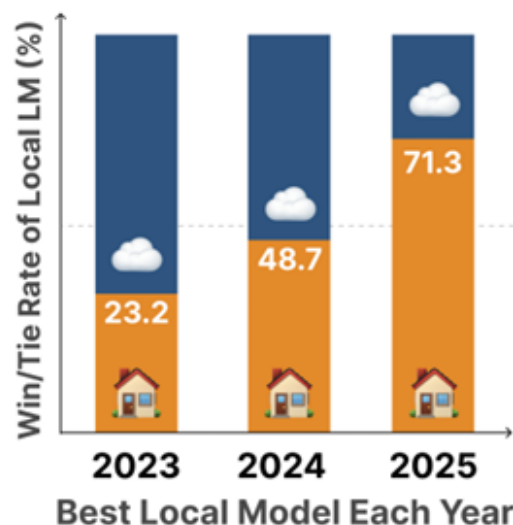
Pin a version, inspect behavior, and fine-tune models for a narrow task.

No API calls / Scale without worrying about token costs

Local Intelligence Efficiency

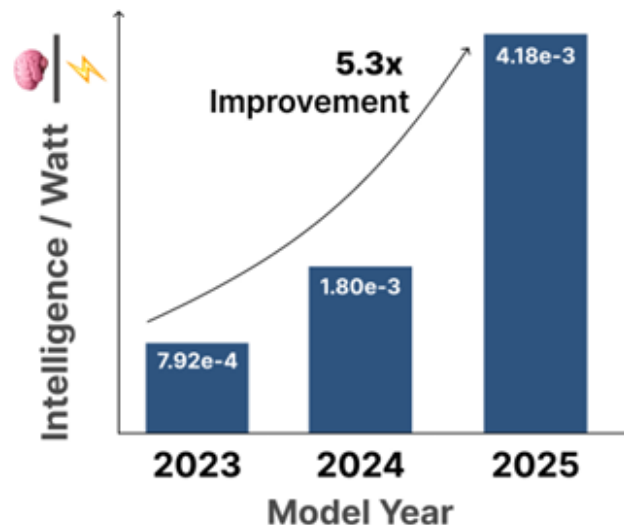
Local LM Intelligence

3.1x improvement over 2 years



Local Intelligence Efficiency

compounded growth from models & accelerators



Clem Delangue 🧐 [in](#) • Following
Co-founder & CEO at Hugging Face
2mo • 🌐

A study from [Stanford University](#) showed that 71.3% of chatgpt queries could be accurately answered by a local model. I suspect a major part of enterprise AI workloads could be run locally too for free (compared to the massive costs of frontier API cost).

Also, it reduces the risk of these workloads being taken away from you because you own the models instead of renting them - which sounds like a good idea these days haha.

That's why we're introducing the ability for everyone to filter AI models on [Hugging Face](#) based on your local hardware.

For me, there are 800k+ public models that fit on my M5 24GB and that I can use easily thanks to llamacpp.

Let's go local AI!

[Intelligence per Watt: Measuring Intelligence Efficiency of Local AI](#)

For local AI deployment, the top-performing open-weight models in September 2026 that meet or exceed ~73% benchmark scores are Qwen3.6-27B, Gemma 4 31B, and Kimi K2.6 — all running well on a single [RTX 4090 \(24 GB VRAM\) at Q4 quantization](#).

Three separate choices

Choice	What to verify	Example
Execution location	Where do inference and data processing run?	On-device, LAN, hosted
Model access	Can we download and run the weights?	Official model repository
Usage rights	What do the exact licenses allow?	Model, runtime, assets

“Runs locally” does not tell you what you may redistribute or commercialize.

Local, cloud, or a deliberate hybrid

Pattern	Works well when...	Main cost
On-device	The task is bounded and must work offline	Memory, battery, thermal limits
Hosted	Quality or scale exceeds the device budget	Network and service dependency
Hybrid	Some tasks justify an explicit cloud fallback	Routing and data-policy complexity

Workshop default: local core, visible failure, no silent cloud fallback.

Memory: weights are only the beginning

4B parameters × 4 bits ÷ 8 = 2 GB

Ideal raw weights only, in decimal GB

16-bit weights	8-bit weights	4-bit weights
8 GB	4 GB	2 GB

Peak memory also includes context state, workspaces, other models, the app and OS.

Where to look, and what to trust

Look here	Use it for	Check before trusting
Hugging Face / publishers	Task, license, context, limitations	Exact model and revision
Ollama / llama.cpp	Supported formats and hardware	Version and model compatibility
Quantized repositories	Smaller deployable artifacts	Upstream source and quantization
Benchmarks + your own cases	Shortlist, then choose	Same task and test conditions

A shortlist for this session

Candidate	Role to test	Published license
Qwen3.5 2B / 4B	Text and image tasks	Apache 2.0
Gemma 4 E2B	Text, image and audio input	Apache 2.0
LFM2.5 1.2B Instruct	Compact text assistant	LFM Open License 1.0
Qwen3 Embedding 0.6B	Semantic retrieval	Apache 2.0
Whisper tiny / base	Speech transcription	MIT

A model name is not a RAM estimate. Check total weights, encoders and actual artifact.

The model-fit contract

Questions to answer

What does the user need to finish?

Which inputs and languages matter?

What are the memory and time limits?

What should happen when it fails?

A useful result on your device matters more than a general leaderboard rank.

Example: document Q&A

20 local documents, English first

Answer with a source passage

16 GB laptop, one request at a time

Target <10 s. Abstain if unsupported.

Hardware lanes are starting points

Device lane	First experiment	Adjustment if it struggles
8 GB laptop, CPU	0.8–2B or one specialist model	Short context, batch work, smaller model
16 GB laptop	2–4B, one model loaded at a time	Unload ASR / embeddings before generation
32 GB or useful GPU	4–9B and larger task inputs	Measure VRAM / RAM and sustained latency
Phone / edge computer	Model with verified native-runtime support	Test battery, thermals and offline startup

Planning heuristics, not fit guarantees. Begin at 4K context and measure peak memory.

A 20-case bake-off

10 typical + 5 difficult + 5 “should abstain”

Quality	Speed	Resources	Reliability
Task success Source / field accuracy	Cold start Warm p50 and p95	Peak RAM / VRAM Battery or thermals	Schema validity Abstention / retries

Same cases, settings and device. Twenty cases screen candidates. They do not certify quality.

Model type determines the surrounding app

Model type	Model output	App components it needs
Embedding / reranker	Vectors / relevance scores	Chunk store, index, retrieval checks
Text generation	Text or structured fields	Context builder, validator, UI
Speech recognition	Transcript and timestamps	Audio capture, segmentation, correction
Vision-language	Image-grounded text / fields	Resize / crop, schema, review
Speech synthesis	Audio	Playback, buffering, cancellation

Runtime choice follows deployment

Deployment	Candidate runtime	Integration shape
Laptop workshop	Ollama	Local HTTP API
Native desktop / edge	llama.cpp	Library or local server
Apple silicon	MLX-LM	Python on Apple silicon
Browser	ONNX Runtime Web	Model in a browser worker
Mobile app	LiteRT-LM	Native inference module

Model format, quantization and runtime version must match. A conversion alone is not enough.

The app owns the workflow

Layer	Responsibility	Common implementation
Interface	Select input, show progress, review output	Local web UI or desktop shell
Application	Prepare context, queue work, enforce limits	Small Python / TypeScript service
Inference	Load model and generate / embed	Local runtime with pinned model
Result + storage	Validate, attach evidence, save / delete	Typed schema, SQLite, local files

One model call is a component. Input handling and verified output make it an app.

A dependable local prototype

Before the model call

Local data path and loopback binding

Input limits, cancellation and timeout

Pinned model + runtime manifest

Pre-downloaded offline dependencies

After the model call

Schema and domain validation

Evidence beside generated claims

Human review before file writes

Clear error, retry and delete controls

Three builds, three different pipelines

Build	Input	Useful output
1 Ask my documents	A small local document folder	Answer with a source passage
2 Meeting notes	A short recorded audio clip	Editable actions with timestamps
3 Receipt to spreadsheet	One receipt image	Reviewed fields, then CSV

Each walkthrough includes a model choice, implementation path and acceptance test.

Build 1: Ask my documents

**“What did we decide,
and where is the source?”**

Scope: 20 text documents

Qwen3 Embedding 0.6B

Qwen3.5 2B

Local UI + SQLite + NumPy



Document Q&A architecture

Stage	Operation	Artifact / contract
1 Index once	Parse, chunk, embed	Text + vector + document/page ID
2 Retrieve	Embed question, rank chunks	Top 4 passages with stable IDs
3 Answer	Supply evidence to the text model	Answer + source IDs, or abstention
4 Review	Resolve citations beside the answer	Clickable original passage

No evidence found? Show “I could not find this in your documents.”

The retrieval-to-answer core

```
E = client.embed(model=EMBED, input=chunks).embeddings
q = client.embed(model=EMBED, input=query_prompt).embeddings[0]
E, q = normalize_rows(E), normalize(q)
keep = (E @ q).argsort()[-4:][::-1]
evidence = join_with_source_ids(chunks, ids, keep)

r = client.chat(model="qwen3.5:2b", think=False,
  messages=[{"role": "user", "content":
    "Use only this evidence. Cite source IDs. "
    "If unsupported, say you do not know.\n"
    + evidence + "\nQuestion: " + question}])
```

Teaching excerpt: helpers handle normalization, IDs and prompt size. Full API setup is in the appendix.

Document Q&A: the first acceptance test

Test	Proposed workshop gate	If it fails
Retrieval	Gold source in top 4 for $\geq 9/10$ questions	Inspect parsing and chunks
Grounding	Every demo citation supports its claim	Shorter context, explicit evidence IDs
Unknown question	Abstain on $\geq 4/5$ unsupported cases	Add missing-evidence examples
Handoff	Another person can index and query offline	Package launcher + sample folder

Demo moment: ask about a fact that is deliberately absent from the folder.

Build 2: Meeting notes

“What did we agree to do?”

Scope: one recorded clip

whisper.cpp + Whisper base
Qwen3.5 2B

Editable transcript and action list



Meeting notes architecture

Stage	Operation	Artifact / contract
1 Prepare	Import clip, convert to mono 16 kHz WAV	Consented recording
2 Transcribe	Run Whisper locally	Timestamped transcript segments
3 Extract	Use text model on the transcript	Action, owner?, due?, evidence time
4 Review	Correct transcript and proposed actions	Local Markdown / JSON export

Speech recognition does not identify owners automatically. Unknown fields stay empty.

Audio in, timestamped text out

```
ffmpeg -i meeting.m4a -ar 16000 -ac 1 \  
-c:a pcm_s16le meeting.wav
```

```
./build/bin/whisper-cli \  
-m models/ggml-base.en.bin -f meeting.wav \  
-oj -of transcript
```

```
# Extract actions with the local text model  
{  
  "action": "Send the draft agenda",  
  "owner": null, "due": null,  
  "evidence_time": "00:32"}  
}
```

Illustrative output contract. Missing owner or due date stays null.

Meeting notes: the first acceptance test

Test	Proposed workshop gate	If it fails
Critical facts	Preserve $\geq 9/10$ labeled facts	Compare against transcript / audio
Action evidence	Every action has a matching timestamp	Extract fewer, shorter actions
Unknown fields	No invented owners or dates in demo set	Use null and human correction
Device fit	Record latency + peak memory	Transcript first, summary later

Demo moment: a noisy clip with an action but no named owner.

Build 3: Receipt to spreadsheet

**“Turn this receipt into
a reviewed CSV row.”**

Scope: four fields from one image

Qwen3.5 4B with image input
Schema validation + human review

CSV export after approval



Receipt extraction architecture

Stage	Operation	Artifact / contract
1 Prepare	Orient, crop, resize image	Original + bounded inference image
2 Extract	Vision model with a JSON schema	Merchant, date, currency, total
3 Validate	Check types and domain rules	Valid, missing, or needs review
4 Approve	Show source beside editable fields	User-approved CSV row

Valid JSON can contain the wrong amount. Schema validation and evidence checks serve different jobs.

A typed output the app can verify

```
class Receipt(BaseModel):  
    merchant: str | None  
    date: str | None  
    currency: str | None  
    total: float | None  
  
r = client.chat(model="qwen3.5:4b", think=False,  
    messages=[{"role": "user", "images": ["receipt.jpg"],  
        "content": "Extract visible fields. Use null if unclear."}],  
    format=Receipt.model_json_schema())  
receipt = Receipt.model_validate_json(r.message.content)  
# Validate values, show source, then ask before export.
```

Receipt extraction: the first acceptance test

Test	Proposed workshop gate	If it fails
Field accuracy	≥36/40 normalized fields correct	Inspect errors by field and layout
Unreadable inputs	3/3 route to null or review	Improve crop, OCR baseline, abstention
Critical values	Report total + currency errors separately	Require explicit correction
Export	0 CSV writes before user approval	Gate export in application code

Demo moment: crop away the total and watch whether the app admits it is missing.

Your team's first slice

- 1 Our user needs...
- 2 The smallest useful input and output are...
- 3 Our device, model and runtime are...
- 4 We will call it working when...

Choose a lane. Pair up. Write one acceptance test before building.

The first hour of building

Build time	Milestone
0–10 min	One local model call works on the target device
10–30 min	One input produces a usable output
30–50 min	Evidence, validation and failure behavior work
50–60 min	Run test cases and prepare a one-minute demo

Demo: user problem, happy path, failure case, measured device behavior.

Appendix: laptop setup

```
ollama pull qwen3.5:2b
ollama pull qwen3-embedding:0.6b # document lane
ollama pull qwen3.5:4b           # receipt lane
ollama show qwen3.5:2b
ollama list
```

```
python -m pip install ollama pydantic numpy
```

```
from ollama import Client
from pydantic import BaseModel
client = Client(host="http://127.0.0.1:11434", timeout=120)
```

Set OLLAMA_NO_CLOUD=1 for the Ollama service, restart, then test with networking off.

Appendix: Whisper setup

```
git clone https://github.com/ggml-org/whisper.cpp.git
cd whisper.cpp
cmake -B build
cmake --build build -j --config Release
sh ./models/download-ggml-model.sh base.en

./build/bin/whisper-cli \
  -m models/ggml-base.en.bin \
  -f samples/jfk.wav -oj -of transcript
```

Unix-like shell example. Use base for multilingual audio. Install FFmpeg separately.

Appendix: model comparison worksheet

Record	Candidate A	Candidate B
Model + revision + quantization	Record	Record
Runtime, device, context / output limits	Record	Record
Task successes / cases + critical failures	Measure	Measure
Cold start + warm p50 / p95 latency	Measure	Measure
Peak RAM / VRAM + offline result	Measure	Measure

Select the smallest candidate that passes the task and device requirements.

Appendix: model cards and licenses

[Qwen3.5 2B: text and image candidate](#)

[Qwen3.5 4B: receipt extraction candidate](#)

[Qwen3 Embedding 0.6B: retrieval candidate](#)

[Gemma 4 E2B: multimodal alternative](#)

[LFM2.5 1.2B Instruct: compact text alternative](#)

[whisper.cpp: speech runtime and model instructions](#)

Links and further source citations are also in the relevant slides' speaker notes.

Appendix: runtime and integration docs

[Ollama: local-only settings, API and device FAQ](#)

[Ollama: structured outputs and validation examples](#)

[Ollama: image input and vision API](#)

[llama.cpp: supported formats and native inference](#)

[MLX-LM: language models on Apple silicon](#)

[ONNX Runtime Web: browser execution](#)

[LiteRT-LM: on-device generative inference](#)

Open-Source / LOCAL MODELS (1/3)

Open-source Models

- Search and Retrieval: [Liquid AI Announces Multi-Year Partnership with Shopify to Bring Sub-20ms Foundation Models to Core Commerce Experiences | Liquid AI](#)
- Embedded In-Car Intelligence: [Mercedes-Benz and Liquid AI Partner to Scale Embedded In-Car Intelligence in North America](#)
- LLM
 - Liquid.ai: [Liquid Foundation Models | Liquid AI](#)
 - Qwen: [QwenLM/Qwen3.6: Qwen3.6 is the large language model series developed by Qwen team, Alibaba Group.](#)
 - GLM: [zai-org/GLM-4.5: GLM-4.5: Agentic, Reasoning, and Coding \(ARC\) Foundation Models](#)
 - MiniMax: [MiniMax M3 - Coding & Agentic Frontier, 1M Context, Multimodal | MiniMax](#)
 - DeepSeek: [deepseek-ai/DeepSeek-V4-Pro · Hugging Face](#)
 - Mistral: [mistralai/Mistral-Medium-3.5-128B · Hugging Face](#)
- VLA – Robotics Foundation:
 - [Physical-Intelligence/openpi](#)
 - [NVIDIA/Isaac-GR00T: NVIDIA Isaac GR00T N1.7 - A Foundation Model for Generalist Robots.](#)
 - [LeRobot · Hugging Face](#)
 - [Post-Training Isaac GR00T N1.5 for LeRobot SO-101 Arm](#)

Open-Source / LOCAL MODELS (2/3)

Open-source Models

- Text-to-Speech and End-to-End Audio
 - Qwen3-TTS – Voice Cloning and TTS: <https://github.com/QwenLM/Qwen3-TTS>
 - Voice Cloning and TTS: [ResembleAI/chatterbox-turbo · Hugging Face](#)
 - Ultra Light Weight TTS: <https://supertonic3.github.io/>
 - End-to-End Audio (ASR/LLM/TTS): [LiquidAI/LFM2.5-Audio-1.5B · Hugging Face](#)
- Curated List: <https://www.bentoml.com/blog/exploring-the-world-of-open-source-text-to-speech-models>
- Music Foundation Model
 - ACE-Step-1.5: [ace-step/ACE-Step-1.5: Local music generation model](#)
 - [HeartMuLa: Music Foundation Models](#)
 - [levo-demo/LeVo](#)
 - [Music Understanding: Paper page - Advancing the Foundation Model for Music Understanding](#)
 - [MusicGPT: rubeniskov/MusicGPT · Hugging Face](#)
 - [ACE-Step 1.5 AI Music Generator | Create Royalty-Free Songs](#)

Open-Source / LOCAL MODELS (3/3)

Open-source Models

- Interactive Humanoid Video Generation
 - [FlowAct-R1: Towards Interactive Humanoid Video Generation](#)
- Virtual Try-on
 - [Voost: https://nxnai.github.io/Voost/](https://nxnai.github.io/Voost/)
- Agentic Video Production System
 - [calesthio/OpenMontage: World's first open-source, agentic video production system. 12 production pipelines, 100+ tools, 700+ agent skill and production-knowledge files. Turn your AI coding assistant into a full video production studio.](#)
- 2D Image -> 3D Mesh:
 - <https://microsoft.github.io/TRELLIS/>
- Curated List of Open-Source Model*: [BayAreaKGroup/OpenModel: Curated List of Open-Source Models - LLM, VLM, Music, Image, Video](#)